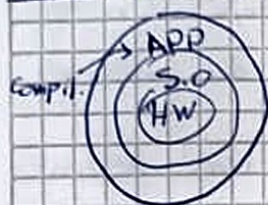


Clasif. de la computacion moderna

- Computadores personales (PC): Optimización de la red. Precio/rendimiento para un unico usuario
- Servidores / Cloud: Orientado a cargas consistentes o muchos tareas pequeñas. Se prioriza la fiabilidad
- Sistemas embebidos: Diseñados para ejecutar una sola app o un conjunto específico de tareas

Capas de software y abstracción



- Software de aplicación (APP): Capa mas externa y abstracta
- Software de sistema (SO): Actúa de intermediario entre APP y HW. Provee servicios comunes
- Hardware

El software se organiza en niveles para ocultar los detalles técnicos del hardware a los programadores de aplicaciones.

Hardware

Cualquier computadora, sin importar su escala, se puede dividir en cinco partes fundamentales:

- Entrada: Como la computadora recibe datos del mundo
- Salida: Como la computadora entrega los resultados
- Ruta de datos: Realiza operaciones aritméticas y lógicas
- Control: Unidad que comanda las partes individuales
- Memoria: Donde se almacenan los programas en ejecución y los datos
 - SRAM (Cache): Memoria de acceso rápido (nóvel). Funciona de buffer. Baja densidad
 - DRAM: Memoria de alta densidad, baja velocidad

} Forman parte del CPU o Nucleo

Arquitecturas de Hardware

Son las variaciones en las anteriores partes fundamentales que permiten clasificar los CPUs según arquitectura. Las mas comunes son:

- ARM (Advanced RISC Machine)
- x86/x64 (Intel y AMD)
- RISC-V (Reduced Instruction Set Computer)
- MIPS (Microprocessor w/o Interlocked pipelined Stages)

- AVR (Microchip / Atmel)

Las diferentes arquitecturas definen una Arquitectura del Conjunto de Instrucciones o ISA. El ISA es la interfaz abstracta entre el hardware y el software de más bajo nivel. Especifica toda la información necesaria y suficiente para escribir un programa para dicha arquitectura. Además, y para que el software sea realmente portátil entre sistemas similares, se introduce el ABI o Interfaz Binaria de Aplicación. Es una combinación del ISA y las interfaces proporcionadas por el software del sistema.

Rendimiento

La definición de rendimiento depende del tipo de computadora.

- Computador personal. Mejor rendimiento es la disminución del tiempo de respuesta o ejecución.
- Servidor. Mejor rendimiento significa mayor ancho de banda o tasa de transferencia.

Si se analiza el rendimiento en función del tiempo

$$\text{Rendim} = \frac{1}{\text{tiempo de ejec.}}$$

Es importante saber qué tiempo hay que usar

- Tiempo transcurrido: Es el tiempo total transcurrido desde que inicia la tarea hasta que finaliza.
- Tiempo de CPU: Es el tiempo que el procesador está ejecutando tareas. Excluye la espera de I/O y el de otros procesos.
- Tiempo de CPU de usuario: El tiempo donde se ejecuta la tarea.
- Tiempo de CPU de sistema: El tiempo que el SO dedica a realizar tareas a petición del programa (gestión de recursos, etc.).

El tiempo de ejecución se puede calcular en función del hardware:

$$t_{\text{cpu}} = \text{Cicl. de clk. para el prog.} \cdot T_{\text{clk}}$$

$$t_{\text{cpu}} = N_{\text{instrucc.}} \cdot \text{CPI} \cdot T_{\text{clk}} \quad \text{CPI: Clocks per Instruction}$$

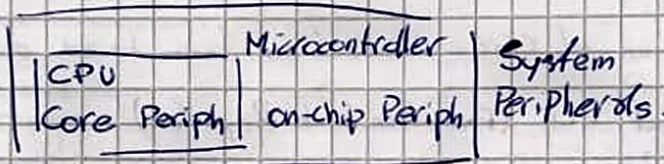
ARM

Advanced RISC Machine es una arquitectura RISC caracterizada por ser licenciada. Existen muchas versiones de ARM, se estudiará ARMv7-M (Cortex-M3). A continuación se explican algunas de sus partes:

- **Decodificador de direcciones:** En todo sistema, el bus de direcciones está conectado a todos los dispositivos simultáneamente. Para evitar que todos (SRAM, FLASH, Periféricos, etc.) respondan al mismo tiempo, el decodificador de direcciones monitorea (generalmente) los MSB, y dependiendo de la secuencia, habilita un dispositivo particular.
- **Input/Output:** También llamados periféricos, son los que permiten que ingresen y egresen datos. Están conectados al bus de direcciones, y tienen su rango de finido. En ARM, para simplificar su interacción, se optó por una topología MMIO (Memory Mapped I/O).

Periféricos ARMv7-M

- **Periféricos externos al núcleo:** Son diseñados por el fabricante.
0x40000000
- **Periféricos Internos:** Diseñados por ARM. Esenciales para el funcionamiento.
0xE0000000



Creación de un programa

- **Compilado:** Proceso por el cual un programa (en C, por ej.) es traducido a ensamblador o código objeto (back-end) por el compilador ^{front-end}.
- **Ensamblado:** Proceso por el cual un programa de back-end es traducido a código máquina por el ensamblador. Se realizan dos "pases" al programa en back-end:

1. Se asignan direcciones de instrucciones, se buscan símbolos (var, func, etc.) y se crea la tabla de símbolos.
2. Se genera el código máquina con la tabla de símbolos.

La salida del ensamblado es un archivo objeto que contiene: Cabecera, segmento de texto (código), segmento de datos estáticos (variables), tabla de símbolos, tabla de relocalización & informe de depuración.

- **Enlazado:** Proceso por el cual se juntan todos los archivos objetos en un archivo en código máquina ejecutable y se asignan direcciones de memoria. También ordena el código resultante. Linker. En μC , el Linker ~~requiere~~ requiere que se especifique el mapa de memoria (.ld)

Como extra, en μC se necesita un programa llamado startup, que se escribe en C o en ASM, y se encarga de preparar el hardware y comenzar la ejecución del programa.

NVIC

Bloque de hardware dentro del núcleo que centraliza la gestión de excepciones e interrupciones.

Una excepción es un evento no programado que rompe el flujo normal de ejecución. Son como funciones, pero se llaman en momentos no previstos. Las excepciones pueden ser causadas por hardware o software. Se pueden clasificar según sincronía y origen.

- Sincronicas: Ocurren en el mismo punto de ejecución.
- Asincronicas: Ocurren de forma impredecible.
- Internas: Generadas por la lógica del CPU.
- Externas: Generadas por eventos fuera del CPU.

Algunas excepciones típicas:

I/O Device Request, Invoke OS Service, Undefined Instruction, Hardware malfunction, Arithmetic Overflow.

Cuando ocurre una excepción, el hardware realiza tres tareas de forma atómica:

1. Guardar el PC en el ELR (Exception Link Register)
2. Registrar la causa.
3. Salto al handler de la excepción.

El registro de la causa puede ser a un registro específico cuando el punto de entrada es el mismo para todas las excepciones, o si las interrupciones son vectorizadas, el punto de entrada varía según la excepción.

El NVIC está caracterizado por ser vectorizado y tener Pre-emption (Interrumpir una interrupción basada en su prioridad). Las interrupciones pueden tener tres estados posibles.

- Disable/Enable: Posibilidad de generar interrupciones.
- Pending: Solicitada pero pendiente de ejecución.
- Active: Ejecutando interrupción.

Periféricos

Todo microcontrolador se basa en una estructura jerárquica de niveles.

1. Capa de infraestructura: Encendido del dispositivo.
2. Capa de interfaz: Conectar el periférico con el mundo exterior.
3. Capa de protocolo: Configurar lo particular de cada dispositivo.
4. Capa operativa: El dispositivo arranca, el software pasa a gestionar el perif.

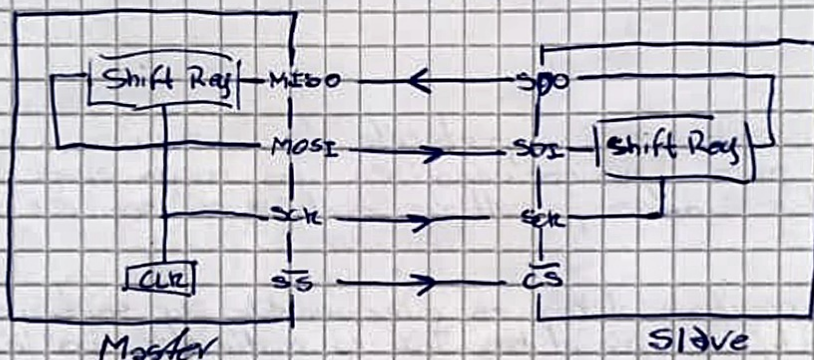
Para la placa de desarrollo Bluepill, la secuencia de activaciones la sig:

1. Capa de Infraestructura: Habilitación de señales de reloj con el periférico Reset & Clock Control (RCC). La bluepill tiene dos buses de periféricos, el clock de cada periférico se encuentra asociado a su bus (APB 1/2).
2. Capa de interfaz: Activar el puente físico entre el dispositivo y los pines (GPIO) y remapear pines si es necesario (AFIO).
3. Capa de protocolo: Configuración particular del periférico.
4. Capa Operativa: Habilitación del periférico.

SPI

Serial Peripheral Interface es un protocolo maestro esclavo, y funciona de maestro simple. El bus es full duplex de 4 líneas de señal más masa. La interfaz física está compuesta por:

- SCK: Serial Clock
- MOSI/SDO: Master out Slave in
- MISO/SDI: Master in Slave out
- SS: Slave Select. Pueden existir n SSs como n dispositivos en el bus.



Para comenzar una transmisión de datos, el proceso es el sig:

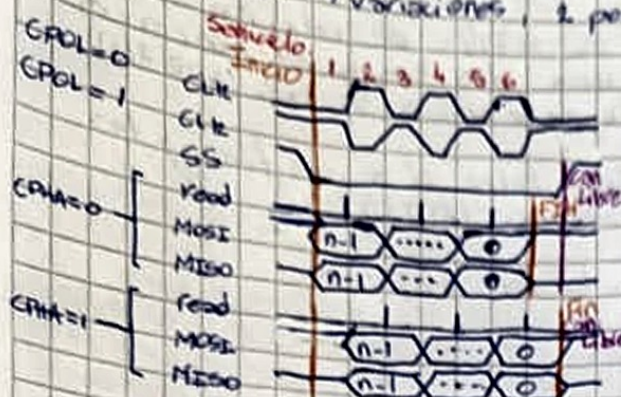
1. El master selecciona al slave con un "0" en SS.
2. Esperar a que el esclavo se inicialice.
3. El master genera el clock, iniciando la comunicación.
4. Durante cada ciclo, el master envía 1b, el esclavo lo lee y envía el mismo bit para que el maestro lo lea.

No existe un orden establecido para los bits, ni tampoco un límite de longitud.

Existen dos formas de conectar esclavos al bus SPI:

- Esclavos independientes: MOSI, MISO y CLK se conectan en paralelo. Se conecta 1 SS por esclavo.
- Daisy Chain: MOSI al esclavo 1 y MISO al esclavo n . CLK en paralelo. El resto de los esclavos se conectan MISO con MOSI. 1 SS para todos. No es posible recibir datos.

Según donde se realice la lectura del dato en un ciclo de reloj, se pueden encontrar 4 variaciones, 2 por fase y 2 por polaridad



CPOL simplemente cambia la polaridad del clock

CPHA=0 utiliza el 1er semiciclo del clock para leer datos y el 2do para el corrimiento. Al final, SS debe subir y bajar.
CPHA=1 utiliza el 2do semiciclo del clock para leer datos y el 3ro para el corrimiento. Al final, SS no requiere subir y bajar

Ventajas

- Full-Duplex
- Driver Push-Pull, alta velocidad
- No hay límite de bits por mensaje
- No requiere elementos externos
- Pocas líneas comparado al paralelo
- Señales unidirecc. (permite arbitraje y colisión)
- No limitado por clock
- Implementar simple de software

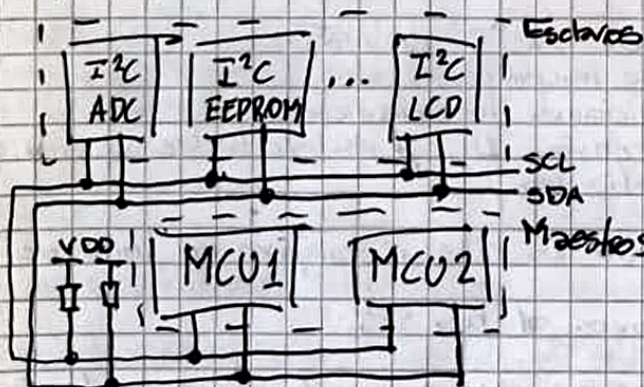
Desventajas

- Requiere más pines que otros protocolos
- Requiere señales dedicadas para direccionar (SS)
- Sin control de flujo por parte del esclavo
- Sin reconocimiento del esclavo o error de detección
- Típicamente un master solo
- No hay estándar formal
- Interrupciones deben ser implementadas de manera externa
- No permite hot-plug

I2C

Inter Integrated Circuit es un protocolo de comunicación serie sincrónico utilizado para comunicaciones en una misma placa. El protocolo I2C permite múltiples maestros y múltiples esclavos con solo dos señales y una masa.

Cada dispositivo conectado al bus es direccionable por software enviando su dirección o identificador por el bus. I2C es multimaster, por lo que implementa arbitraje y detección de colisiones. Las transmisiones de datos son fijas de 8b, con una tasa máxima de 3.4 Mbps en modo bidireccional o 5 Mbps en modo unidireccional.



Señales de datos

- SCL (System Clock): Solo maestros
- SDA (System Data): Bidireccional
- GND

Ambos SCL y SDA son Open Drain, por lo que su estado alto debe ser obtenido por pull-up o similar

Estados del bus

- Bus libre: SCL y SDA en alto
- Master inicia comunicación: SCL en alto y SDA en bajo



Formato de la trama

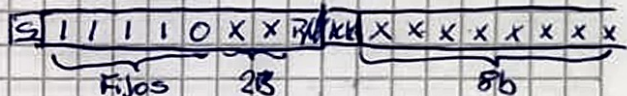
- Longitud: 1B (MSB primero)
- Acknowledge: luego del dato. Quien recibe el dato baja la línea SDA indicando éxito.
- Estiramiento del reloj: Si el slave se demora y tiene que pausar la transmisión/recepción, mantiene el SCL en bajo.
- Antes de enviar el dato se debe especificar el destinatario y tipo de operación (read/write). Es un dato "especial". El destinatario es de 7b y el tipo de operación es el 8º bit LSB.

Posibles secuencias de transmisión

- Maestro transmite al esclavo
 - M → Dirección S, modo Write
 - S → ACK
 - M → DATO
 - S → ACK
 - Repetir hasta acabar datos
 - M → STOP
 - Esclavo transmite datos
 - M → Dirección S, modo Read
 - S → ACK
 - S → DATO
 - M → ACK
 - Repetir hasta acabar datos
 - M → NACK, STOP
- El Master SIEMPRE controla SCL

- Combinado
 - M → Dirección S1
 - S1 → ACK
 - Ocurre lo que tenga que ocurrir
 - M → STOP y START o solo START
 - M → Dirección S2
 - S2 → ACK
 - ...

- Direcciones de 10 bits
 - Permiten expandir el # de esclavos
 - Direcciones de 7 y 10 bits pueden compartir bus.
 - De los 10b, 2b se transmiten después del start (+R/W) y 8b en la subsecuente transmisión



- Modo HS
 - M → envía 00001xxx
 - M espera a que se libere SCL para cambiar a modo HS
 - M → Dirección S
 - Ocurre lo que tenga que ocurrir
 - M → STOP, se sale de modo HS

Direcciones reservadas:

- 0000 0001 llamada genl
- 0000 0000 start
- 0000 001x CBUS
- 0000 010x otro tipo de bus
- 0000 100x reservado
- 0000 1xxx modo HS
- 1111 1xxx ID
- 1111 0xxx modo 10b

UART

Periférico de comunicación serial asincrónico. Utilizado en protocolos RS-232 y RS-485.

RS-232

En su modo más simple solo requiere 2 líneas, Tx, Rx y GND. La norma provee dos dispositivos conectados al bus, Data Terminal Equipment (PC) y Data Communication Equipment (Modem).

La trama tiene la sig. estructura

- Comienza con un bit en 0 (start)
- Se envían los datos comenzando por el LSB
- La palabra es de longitud fija (5 a 8 b)
- Finaliza con un bit en 1 o más (stop)
- Opcionalmente, antes del stop puede haber un bit de paridad



RS-232 tiene muchos señales más que le permiten tener más control

Nombre	Descripción
DTR	Data Terminal Ready (DTE)
DCD	Data Carry Detect (DCE)
DSR	Data Set Ready (DCE)
RI	Ring Indicator (DCE)
RTS	Request to Send (DTE)
RTR	Ready to Receive (DTE)
CTS	Clear to Send (DCE)
TxD	Transmitted Data
RxD	Received Data
GND	Common Ground
PG	Protective Ground

Se pueden usar de dos formas:

- Flow Control
 - RTS/RTR=0 → listo p/raib. (DTE)
 - CTS=0 → listo p/raib. (DCE)

• Simple Mode

- RTS=0 → listo p/raib. (DTE) y
- CTS=0 → listo p/raib. (DCE)

señales de HW Handshake

FreeRTOS

Es un SO de tiempo real diseñado para microcontroladores y pequeños microprocesadores. Está caracterizado por:

- Preemptive Scheduling: El planificador le da el control a la tarea de mayor prioridad que esté lista.
- MicroKernel: Núcleo optimizado. 4-9 kB
- Mecanismos de sincronización: Dispone de Semáforos, mutex, colas, grupos de eventos y timers
- Portabilidad: Escrito en C con pocas líneas en ASM

FreeRTOS utiliza notación húngara para variables y funciones. En esta notación, se antepone un prefijo con el tipo de dato del elemento

• c → char/int8_t ; s → int16_t ; l → int32_t ; u → unsigned ; p → puntero ; X → BaseType_t y otros.
v → void ; prv → func. privados.

FreeRTOS utiliza las variables de longitud fija por cuestiones de portabilidad. Además, agrega los sig. tipos de datos:

- TickType_t: Usado para almacenar ticks. Puede ser de 16b o 32b.
- BaseType_t: Es el tipo de dato más eficiente dada la arquitectura.

Las tareas de FreeRTOS son funciones que nunca retornan. Se implementan con un bucle interno, tienen su propio stack para variables locales y contexto de ejecución. El único requisito es que tengan el sig. prototipo:

```
void mytask(void *pvParameters);
```

De la misma forma que al final del main se agrega un return 0, para las tareas se agrega vTaskDelete(NULL), indicando que la propia tarea se elimina. Es un "fail-safe", ya que el bucle de la tarea nunca debería terminar. Las tareas tienen dos estados principales:

- Running: La tarea tiene el control del CPU (solo puede haber una).
- Not Running: La tarea está esperando su turno de ejecución.

Para crear tareas se usa la función xTaskCreate(...), cuyos argumentos son (en este orden):

1. TaskFunction_t: Puntero a la función tarea
2. const char *: Nombre (para depuración)
3. uint16_t: Tamaño del stack (en palabras)
4. void *: Parámetros (argumentos) para la tarea
5. BaseType_t: Prioridad (0 es la más baja)
6. TaskHandle_t: Handler de la tarea

Siempre se debe comprobar lo que haya devuelto la creación de la tarea:

pdPASS → Tarea creada ; pdFAIL → Tarea no creada

La prioridad de la tarea le da la pauta al scheduler de cuál ejecutar primero cuando el SO obtiene el control del CPU. ~~El SO también~~ También, gracias a la "tick interrupt" el CPU obtiene el control del CPU para gestionar tareas. La frecuencia del tick interrupt se define en configTICK_RATE_HZ.

Para que el scheduler pueda entregar el control del CPU a otras tareas de menor prioridad, es importante que la tarea no realice esperas activas (esperar a un cambio de estado o dato de un periférico). FreeRTOS implementa alternativas a estas esperas activas donde las tareas le indican al SO "No tengo nada que hacer", cediendo el control del CPU. Cuando ocurre que la tarea en ejecución "No tiene nada que hacer", pasa a estado NotRunning, y dependiendo del tipo de espera que se haya utilizado, puede estar en los sig. subestados:

- Blocked: La tarea está esperando un evento.
 - Evento temporal: espera que un lapso de tiempo transcurra (delay())
 - Evento de sincronización: Espera que otra tarea finalice o llegue una interrupción (llegada de datos)

- Suspend: La tarea está congelada y no será ejecutada por el scheduler hasta que no esté más suspendida.
- Ready: La tarea está lista para ejecutarse.

Es importante que las tareas implementen estas esperas de FreeRTOS para evitar que una tarea utilice todo el tiempo del CPU, generando que las otras tareas queden en estado "starved" (sin posibilidad de ejecución).

A continuación se detallan algunas funciones para que las tareas entren en los subestados de NotRunning:

- Blocked por evento temporal: `void vTaskDelay(TickType_t xTicksToDelay);`

Esta función pone en Bloqueo temporal a la tarea por la cantidad de ticks especificados. Se puede usar la macro `pdMS_TO_TICKS()` para pasar de ms a ticks. La macro funciona solo si `configTICK_RATE_HZ` es menor a 1kHz.

La func. anterior presenta una espera relativa. FreeRTOS implementa la sig. función para hacer esperas absolutas:

```
void vTaskDelayUntil(TickType_t *pxPreviousWakeTime,
                    TickType_t xTimeIncrement);
```

Donde `pxPreviousWakeTime` es el puntero a la variable que almacena el $t=0$ y `xTimeIncrement` la espera deseada. De esta forma, se puede hacer que las tareas se ejecuten cada un intervalo fijo de tiempo, en vez de tener variaciones según cuanto tarde la tarea en ejecutarse. `pxPreviousWakeTime` es actualizado con el número de ticks en el que el SO devuelve el control a la tarea.

- Suspended: `void vTaskSuspend(xTaskHandle);`
`void vTaskResume(xTaskHandle);`

FreeRTOS implementa una tarea de prioridad 0 llamada Idle. En esta tarea se realizan procesos de gestión de recursos, y es la tarea que se ejecuta cuando no hay más tareas listas para ejecutar. FreeRTOS permite añadirle al Idle una tarea definida por el usuario llamada `IdleHook`, la cual se va a ejecutar con la misma.

Así como se crean las tareas en tiempo de ejecución, se pueden borrar y cambiar su prioridad.

```
void vTaskPrioritySet(TaskHandle_t pxTask,
                    UBaseType_t uxNewPriority);
```

```
UBaseType_t uxTaskPriorityGet(TaskHandle_t pxTask);
```

```
void vTaskDelete(TaskHandle_t pxTask); → pxTask = NULL → se borra la tarea q' llamó
```


Para comunicar datos entre tareas sin usar variables globales, FreeRTOS implementa un sistema de comunicación llamado colas. Las colas son buffer FIFO, con una longitud por dato fija y un número finito de elementos. El uso de colas en tareas tiene los siguientes ventajas:

- Comunicación segura: Evita condiciones de carrera.
- Desacoplamiento de tareas: Una vez que el dato está en la cola, la tarea que lo envía puede descartar el mismo.
- Diseño eficiente y orientado a eventos: Bloqueos de r/w sin ~~uso~~ del CPU.
- Gestión de memoria simple: Todo lo gestiona FreeRTOS.

Por defecto, los datos de las colas son almacenados por copia.

A continuación se detalla la API de las colas.

- Creación de colas: `QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t uxItemSize);`
- Envío a Cola: `BaseType_t xQueueSendToBack(UBaseType_t xQueueSendToFront(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);`
 ✓ `BaseType_t xQueueSendToFront(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);`
 pdPASS → OK Tiempo de espera — si la cola está llena
 or QUEUE_FULL
- Recepción de Cola: `BaseType_t xQueueReceive(QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);`
 pdPASS → OK
 or QUEUE_EMPTY → No se recibió nada
- No de elem.: `UBaseType_t uxQueueMessagesWaiting(QueueHandle_t xQueue);`

FreeRTOS no implementa ningún sistema o API para gestionar interrupciones. Si propone ciertas herramientas y características que debe tener una interrupción. Las interrupciones, al ser gestionadas por hardware, siempre van a obtener el control contra las tareas. Esto significa un problema, porque el SO no tiene control sobre lo que se ejecuta, rompiendo la predictibilidad de RTOS. Como pauta general, los ISRs deben ser lo más cortos posibles, para devolverle el control al SO.

Existe una API ~~segura~~ para ISRs. Esta integra algunos cambios, y no pone a disposición todas las funciones de FreeRTOS, debido a que un ISR no es una tarea (no se puede bloquear). Para diferenciarlas, se les añade `FromISR()` al final del nombre. Todas las funciones de esta API tienen un parámetro extra, `*pxHighPriorityTaskWoken`. Este parámetro sirve para indicarle al kernel, cuando termina el ISR, si se ha despertado una tarea de prioridad más alta que la de la tarea que se estaba ejecutando. Es responsabilidad del ~~programador~~ desarrollador indicarle al kernel con la macro `portYIELD_FROM_ISR` o `portEND_SWITCHING_ISR()` al final del ISR con el resultado de `*pxHighPriorityTaskWoken` para forzar el cambio de contexto.

FreeRTOS implementa el sistema de semáforos binarios para bloquear y desbloquear tareas particulares. Esto es útil cuando de la interrupción.

se requiere hacer `processm` extra que va a demandar tiempo pero debe hacerse en el momento. De esta forma, de una ISR, se puede ejecutar una tarea como si fuese la propia ISR, pero comandada por el SO. El uso de semáforos es simple

- Crear un semáforo: `SemaphoreHandle_t xSemaphoreCreateBinary(void)`

- Tomar un semáforo: `BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);`

Usado para "Suscribirse" al semáforo. Esta función solo debe efectuarse en tareas. `xTicksToWait` indica cuánto espera si el semáforo no está disponible

- Dar un semáforo: `BaseType_t xSemaphoreGive(SemaphoreHandle_t xSemaphore);`

Usado para "Dar luz verde" a un semáforo y desbloquear la tarea ^{esperando}. La versión ISR tiene el parámetro extra `pxHigherPriorityTaskWoken`.